Object Oriented Programming in Java

Exceptions in Java

Suhel Hammoud

What Is an Exception?

An **exception** is an event that occurs during program execution that disrupts the normal flow of instructions.

Example of an Exception

```java
int[] numbers = {1, 2, 3};
System.out.println(numbers[5]); // Exception!
```

This causes:

- ArrayIndexOutOfBoundsException
- Because index 5 does not exist

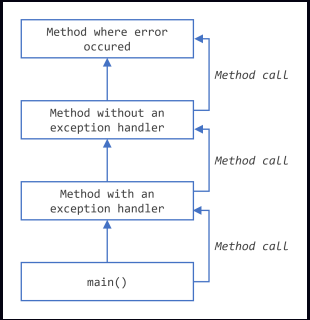
What Happens When an Exception Occurs?

- Java creates an **exception object**
- The object contains information about the error
- Exception object is handed off to the runtime system (throwing an exception)
- The program flow is interrupted
- The runtime system searches for a handler in the ordered list of methods that led to the exception (**call stack**)
- If found then exception is handled
- If not found then the program terminates with an error message

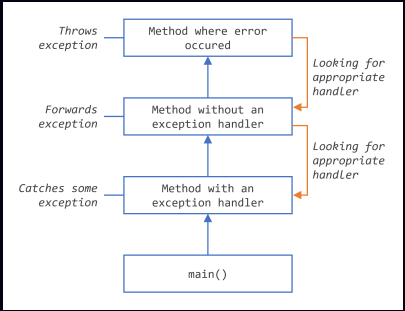
This process is called **exception handling**

Exceptions in Java

Exception call stack



Exception Handler



▒▒▒ Exception Object

An exception object contains:

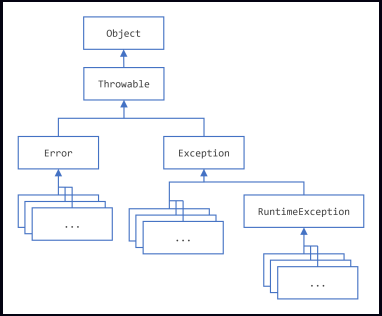
- Type of error
- Location where it occurred
- Stack trace (call history)

Example:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException
```

Exception Hierarchy

All exceptions are part of a class hierarchy.



Errors vs Exceptions

Errors

- Serious problems
- Not meant to be handled
- Example:

```
OutOfMemoryError
```

Exceptions

- Conditions applications may handle
- Recoverable
- Example:

```
IOException
```

Checked vs Unchecked Exceptions

Checked Exceptions

- Checked at compile time
- Must be handled or declared

```
IOException
```

Unchecked Exceptions

- Occur at runtime
- Not required to handle

```
NullPointerException
```

▒▒▒ Checked Exception Example

```
FileReader file = new FileReader("file.txt");
```

Compiler error unless handled:

```
try {
    FileReader file = new FileReader("file.txt");
} catch (IOException e) {
    e.printStackTrace();
}
```

Unchecked Exception Example

```java
String str = null;
System.out.println(str.length()); // NullPointerException
```

- No compile-time error
- Fails at runtime

Why Use Exceptions?

- Separate error-handling code from regular code
- Improve readability
- Handle unexpected situations gracefully
- Prevent program crashes

Catching and Handling Exceptions

Java provides a mechanism to **handle exceptions** so that programs can continue running.

This is done using:

- `try`
- `catch`
- `finally`

The try Block

The `try` block contains code that might throw an exception.

```
try {
    // code that may throw exception
}
```

- Must be followed by at least one `catch` or a `finally`

The catch Block

A `catch` block handles the exception.

```
try {
    // risky code
} catch (ExceptionType name) {
    // handler code
}
```

- Runs only if matching exception occurs
- `name` gives access to the exception object

Example: Basic try-catch

```java
try {
    int[] arr = {1, 2, 3};
    System.out.println(arr[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid index!");
}
```

- Prevents program crash
- Handles the error gracefully

▒▒▒ Multiple catch Blocks

You can handle different exceptions separately:

```
try {
    // code
} catch (IOException e) {
    // handle IO
} catch (SQLException e) {
    // handle DB
}
```

- Only one `catch` block executes
- Order matters (specific → general)

▒▒▒ Catch Order Matters

```
catch (Exception e) { }   // general
catch (IOException e) { } // ❌ unreachable
```

Correct order:

```
catch (IOException e) { }
catch (Exception e) { }
```

▒▒▒ Multi-Catch (Java 7+)

Handle multiple exceptions in one block:

```
try {
    // code
} catch (IOException | SQLException e) {
    System.out.println("Exception occurred");
}
```

- Reduces duplication
- Exception variable is implicitly final

▒▒▒ The finally Block

The `finally` block always executes:

```
try {
    // code
} catch (Exception e) {
    // handler
} finally {
    // always runs
}
```

- Executes whether exception occurs or not
- Commonly used for cleanup

▒▒▒ Example: finally Usage

```java
FileReader file = null;
try {
    file = new FileReader("file.txt");
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // Try to close the file!
    if (file != null) {
        file.close(); // can you see the problem here?
    }
}
```

Ensures resource is closed

▒▒▒ try-with-resources (Java 7+)

Automatically closes resources:

```
try (FileReader file = new FileReader("file.txt")) {
    // use file
}
```

- No need for `finally`
- Resource must implement `AutoCloseable`

▒▒▒ try-with-resources Example

```java
try (BufferedReader br =
        new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

Cleaner and safer resource management

Accessing Exception Information

Exception object provides useful methods:

```
catch (Exception e) {
    System.out.println(e.getMessage());
    e.printStackTrace();
}
```

Common methods:

- `getMessage()`
- `printStackTrace()`

Stack Trace

Stack trace shows:

- Method call history
- Where the exception occurred

Example:

```
java.lang.NullPointerException
    at Main.main(Main.java:5)
```

Throwing Exceptions

In Java, you can explicitly throw exceptions using the `throw` statement.

This allows you to:

- Signal that an error has occurred
- Enforce rules in your program

The throw Statement

Syntax:

```
throw exceptionObject;
```

- Used inside methods or blocks
- Must throw an object of type Throwable

Example: Throwing an Exception

```
if (age < 18) {
    throw new IllegalArgumentException("Age must be at least 18");
}
```

- Creates and throws exception immediately
- Stops normal program flow

What Happens After throw?

- Execution stops at the `throw` statement
- Control is transferred to nearest handler
- If no handler → program terminates

▒▒▒ Throwing Checked Exceptions

Checked exceptions must be declared using `throws`.

```
public void readFile(String fileName) throws IOException {
    if (!Files.exists(Path.of(fileName))) {
        throw new IOException("File not found");
    }
}
```

- Compiler enforces handling
- Must be caught or declared

▒▒▒ The throws Clause

Used in method declarations:

```
public void methodName() throws ExceptionType {
    // code
}
```

- Declares possible exceptions
- Passes responsibility to caller

Example: Using throws

```java
public void process() throws IOException {
    FileReader file = new FileReader("file.txt");
}
```

Caller must handle it:

```java
try {
    process();
} catch (IOException e) {
    e.printStackTrace();
}
```

throw vs throws

throw

- Used inside method
- Throws one exception
- Followed by object

```
throw new Exception();
```

throws

- Used in method signature
- Declares exceptions
- Can list multiple

```
void m() throws IOException
```

Throwing Unchecked Exceptions

Unchecked exceptions do NOT require declaration:

```
throw new NullPointerException("Null value!");
```

- Not checked at compile time
- Often used for programming errors

When to Throw Exceptions

- Invalid method arguments
- Violation of business rules
- Unexpected program state
- Input validation failures

▒▒▒ Creating Custom Exceptions

You can define your own exception classes:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Throw it like:

```
throw new InvalidAgeException("Invalid age");
```

Checked vs Unchecked Custom Exceptions

Checked

- Extend `Exception`
- Must be declared

Unchecked

- Extend `RuntimeException`
- No declaration required

▒▒ Example: Custom Exception Usage

```
public void register(int age) throws InvalidAgeException {
    if (age < 18) {
        throw new InvalidAgeException("Must be 18+");
    }
}
```

Ensures rule enforcement

Best Practices

- Throw specific exceptions
- Include meaningful messages
- Avoid overusing checked exceptions
- Use custom exceptions when needed
- Do not use exceptions for normal flow

Unchecked Exception Controversy

Java distinguishes between:

- Checked exceptions
- Unchecked exceptions

The debate centers on:

- Should exceptions be enforced at compile time?
- Or handled only at runtime?

Advantages of Checked Exceptions

- Forces developers to handle errors
- Makes APIs explicit about failures
- Improves reliability

Encourages careful programming

Disadvantages of Checked Exceptions

- Leads to verbose code
- Can clutter method signatures
- Often handled poorly (empty catch blocks)

Developers may ignore or misuse them

▒▒▒ Advantages of Unchecked Exceptions

- Cleaner, less verbose code
- No need to declare in methods
- Better for programming errors

More flexible design

▒▒▒ Disadvantages of Unchecked Exceptions

- Errors may go unnoticed
- No compile-time enforcement
- Can lead to runtime failures

Requires discipline from developers

Design Philosophy

Checked Approach

- Safer
- Explicit
- Compiler-enforced

Unchecked Approach

- Simpler
- Flexible
- Developer responsibility

When to Use Checked Exceptions

- Recoverable conditions
- External system failures (I/O, network)
- Situations caller can handle

When to Use Unchecked Exceptions

- Programming errors
- Invalid arguments
- Bugs (null references, logic errors)

references:

- https://dev.java/learn/exceptions/